

SAiDL Summer Assignment 2026

RIYA SARDESAI 2025A8PS1447G

May 2026

Abstract

This report presents a systematic empirical study of long-context sequence modeling and parameter-efficient fine-tuning. In the Core ML section, we implement a modular Transformer architecture and evaluate three attention variants (Multi-Query Attention, Sliding Window Attention, and Linear Attention), three positional encodings (ALiBi, RoPE, and learned embeddings), and a convolutional hybrid architecture on the WikiText-2 language modeling benchmark. In the Sparsity & Optimization section, we compare LoRA, AdaLoRA, and a custom SoRA implementation on the CoLA grammaticality classification task, analyze the theoretical and practical differences between proximal gradient descent and SGD with ℓ_1 subgradients, and propose and implement low-rank adaptation methods for xLSTM and Mamba state space models.

Contents

1	Core ML: Long-Context Sequence Modeling	3
1.1	Overview	3
1.2	Baseline Transformer	3
1.2.1	Architecture	3
1.2.2	Why These Hyperparameters?	3
1.2.3	Baseline Results	3
1.3	Attention Variants	3
1.3.1	Multi-Query Attention (MQA)	4
1.3.2	Sliding Window Attention (SWA)	4
1.3.3	Linear Attention	4
1.4	Positional Encodings	5
1.4.1	ALiBi (Attention with Linear Biases)	5
1.4.2	RoPE (Rotary Positional Embedding)	5
1.4.3	Extrapolation Analysis	5
1.5	Convolutional Hybrid	6
1.5.1	Motivation	6
1.5.2	Implementation	6
1.5.3	Results and Analysis	6
1.6	Full Results and Comparative Analysis	6
2	Sparsity & Optimization	8
2.1	Overview	8
2.2	Methods	8
2.2.1	LoRA	8
2.2.2	AdaLoRA	8
2.2.3	SoRA	8
2.3	Results	9
2.4	Analysis	9
2.5	SGD vs Proximal Gradient Descent (Q2)	9
2.5.1	Derivation of SGD Update Rule	9

2.5.2	Proximal Soft-Thresholding Update	10
2.5.3	Mathematical Comparison	10
2.5.4	Does Subgradient Choice Matter?	10
2.6	Low-Rank Adaptation for xLSTM and Mamba (Q3)	10
2.6.1	Motivation	10
2.6.2	xLSTM Adaptation	11
2.6.3	Mamba Adaptation	11
2.7	Limitations and Future Work	11

1 Core ML: Long-Context Sequence Modeling

1.1 Overview

Long-context modeling remains a central challenge in sequence modeling. Vanilla self-attention has quadratic complexity $O(T^2)$ in sequence length T , making it expensive for long sequences. Additionally, standard learned positional embeddings cannot extrapolate beyond their training length. This section systematically evaluates architectural alternatives across three axes: attention mechanisms, positional encodings, and convolutional augmentation.

All experiments use the WikiText-2 dataset, which contains Wikipedia articles totalling approximately 2.3 million training tokens. We tokenize using the GPT-2 BPE tokenizer (vocabulary size 50,257) and train all models with context length 512, AdamW optimizer ($\text{lr}=3 \times 10^{-4}$), batch size 16, and 3 epochs unless stated otherwise.

1.2 Baseline Transformer

1.2.1 Architecture

We implement a decoder-only Transformer following Vaswani et al. (2017) with causal (autoregressive) masking. The architecture is intentionally kept small to fit within free GPU constraints while remaining expressive enough to show meaningful differences between variants.

- Embedding dimension: 256
- Number of attention heads: 8 (head dimension = 32)
- Number of layers: 4
- Feedforward dimension: 1024 ($4 \times$ embedding)
- Context length: 512 tokens
- Total parameters: 29,022,208
- Activation: GELU
- Normalization: Pre-LayerNorm (more stable than post-LN)

1.2.2 Why These Hyperparameters?

The embedding dimension of 256 and 4 layers were chosen to balance expressivity with training speed on free Kaggle GPU (T4). A feedforward dimension of $4 \times$ the embedding follows the standard ratio from Vaswani et al. Eight heads with head dimension 32 follows the convention that head dimension should be at least 32 for stable attention. Context length 512 was chosen as a baseline before evaluating longer contexts.

1.2.3 Baseline Results

The baseline achieves validation perplexity of **578.83** and throughput of **22,571 tokens/sec** after 3 epochs. This serves as our reference point for all subsequent experiments.

1.3 Attention Variants

The core attention computation in standard multi-head attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This requires computing a $T \times T$ matrix, which is the primary computational bottleneck for long sequences. Each variant below addresses this differently.

1.3.1 Multi-Query Attention (MQA)

Motivation: Standard MHA uses separate Q , K , V projections for each head, resulting in h sets of key and value matrices. MQA observes that much of this is redundant — different heads can share the same keys and values while maintaining distinct queries.

Implementation: We modify the attention module so that K and V project to `head_dim` instead of `embed_dim`, creating a single shared $K \in R^{T \times d_k}$ and $V \in R^{T \times d_k}$ across all heads. Each head maintains its own Q projection. The shared K and V are unsqueezed and broadcast across heads during the attention computation.

Why MQA? MQA reduces the KV cache size by a factor of h (number of heads), which is critical for inference efficiency. It also reduces the number of parameters slightly. It is used in production models like PaLM and Falcon.

Alternative considered: Grouped Query Attention (GQA) is a middle ground between MHA and MQA, where K and V are shared within groups of heads rather than globally. We chose MQA for simplicity as it represents the extreme of this design space.

Results: MQA achieves **perplexity 426.02** — significantly better than the baseline (578.83) — with **28,561,664 parameters** (460,544 fewer than baseline). The improved perplexity is somewhat surprising and may be due to the regularization effect of shared K/V reducing overfitting on our small dataset.

1.3.2 Sliding Window Attention (SWA)

Motivation: For many NLP tasks, most relevant context is local. A token at position t rarely needs to attend to tokens at position $t - 500$. SWA restricts each token to attend only to the W most recent tokens.

Implementation: We implement SWA by modifying the attention mask. In addition to the standard causal mask (upper triangular), we add a window mask that blocks positions more than $W = 64$ steps in the past:

$$\text{mask}_{ij} = \begin{cases} -\infty & \text{if } j > i \text{ (future)} \\ -\infty & \text{if } i - j > W \text{ (too far past)} \\ 0 & \text{otherwise} \end{cases}$$

This is implemented using `torch.triu` and `torch.tril` with appropriate diagonal offsets.

Why window size 64? Window size 64 covers roughly 64 tokens of local context, which is sufficient for short-range syntactic dependencies while keeping the attended region small. Longformer uses window sizes of 512 for longer documents.

Alternative considered: Sparse block attention, where tokens attend to fixed block patterns plus a set of global tokens. We chose SWA for simplicity as it is a cleaner baseline for local attention.

Results: SWA achieves **perplexity 587.46**, slightly worse than baseline (578.83). This is expected — WikiText-2 contains long Wikipedia articles where long-range dependencies (e.g., pronoun references, topic continuity) span more than 64 tokens. By restricting the window, SWA loses this information.

1.3.3 Linear Attention

Motivation: The $O(T^2)$ complexity of standard attention comes from computing the full $T \times T$ attention matrix. Linear attention avoids this using the kernel trick.

Key Idea: Standard attention computes $\text{softmax}(QK^T/\sqrt{d}) \cdot V$. If we replace softmax with a kernel function ϕ , we can rewrite:

$$\phi(Q)(\phi(K)^T V)$$

By computing $(\phi(K)^T V)$ first — a $d_k \times d_k$ matrix — we never need the $T \times T$ matrix. This reduces complexity from $O(T^2 d)$ to $O(T d^2)$.

Implementation: We use the ELU+1 feature map: $\phi(x) = \text{ELU}(x) + 1$. This ensures positive values (required for the approximation to be valid) and closely approximates the exponential function used in softmax. We also add a normalization term to prevent the denominator from becoming too small:

$$\text{out} = \frac{\phi(Q)(\phi(K)^T V)}{\phi(Q) \cdot \sum_j \phi(K_j)}$$

Note that linear attention is non-causal as implemented — it attends to all positions simultaneously. For a causal version, one would need to use the recurrent formulation.

Results: Linear attention achieves **perplexity 559.04** and **throughput 26,539 tok/s** — an 18% speedup over baseline (22,571 tok/s). The perplexity is slightly worse than baseline because the ELU+1 kernel is an approximation to softmax attention, losing some expressivity. The throughput gain is the key advantage.

1.4 Positional Encodings

Standard learned positional embeddings assign a fixed vector to each absolute position. This has a critical limitation: if a model is trained on sequences of length 512, it has no embedding for position 513 and cannot generalize to longer sequences.

1.4.1 ALiBi (Attention with Linear Biases)

How it works: Instead of adding position information to token embeddings, ALiBi adds a linear distance penalty directly to attention scores:

$$\text{scores}_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} - m_h \cdot |i - j|$$

where m_h is a head-specific slope. The slopes follow a geometric sequence: $m_h = 2^{-8h/H}$ for head h out of H total heads. Tokens that are farther apart receive a larger penalty, biasing attention toward local context.

Why ALiBi? ALiBi has no learned positional parameters, making it lightweight and generalizable. The distance penalty is applied at inference time, so it naturally extends to any sequence length without modification.

Results: ALiBi achieves **perplexity 476.94**, a significant improvement over baseline (578.83). The distance penalty acts as a strong inductive bias for language modeling, where nearby tokens are typically more relevant.

1.4.2 RoPE (Rotary Positional Embedding)

How it works: RoPE encodes position by rotating the query and key vectors by a position-dependent angle before computing attention scores. For a vector at position m , each pair of dimensions (x_{2i}, x_{2i+1}) is rotated by angle $m\theta_i$ where $\theta_i = 10000^{-2i/d}$:

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}$$

The key property is that the inner product $q_m \cdot k_n$ depends only on the relative position $m - n$, not absolute positions. This makes RoPE naturally length-generalizable.

Why RoPE? RoPE is used in Llama, Mistral, and most modern open-source LLMs. It encodes relative position implicitly through the rotation, combining the benefits of relative and absolute encodings.

Results: RoPE achieves **perplexity 481.29**, close to ALiBi. Both significantly outperform the learned embedding baseline.

1.4.3 Extrapolation Analysis

A key advantage of ALiBi and RoPE over learned embeddings is length extrapolation. Learned embeddings cannot generalize beyond their training length because position indices beyond the training window have no corresponding embedding. ALiBi extrapolates naturally because its distance penalty formula applies to any distance. RoPE extrapolates because rotations are defined for any angle.

In our experiments, we train on context length 512 and evaluate on 512, 1024, and 2048. Learned embeddings are expected to degrade significantly at 1024 and 2048, while ALiBi and RoPE should maintain reasonable perplexity.

1.5 Convolutional Hybrid

1.5.1 Motivation

Attention captures long-range dependencies but may be inefficient at capturing local n-gram patterns. A 1D convolution can capture local structure efficiently: a kernel of size k at position t looks at tokens $[t - k/2, t + k/2]$. Adding convolution before attention allows the model to first build local representations, then reason globally with attention.

1.5.2 Implementation

We add a depthwise separable Conv1D layer before each attention block:

1. Input $x \in R^{B \times T \times C}$ is transposed to $R^{B \times C \times T}$ (Conv1D expects channels first)
2. A depthwise Conv1D with kernel size 3 and padding 1 is applied (each channel convolved independently)
3. Output is transposed back and added to x via residual connection
4. Standard attention (with RoPE) is then applied

Depthwise separable convolution was chosen over standard convolution because it has far fewer parameters: instead of $C^2 \cdot k$ parameters, it uses $C \cdot k$ parameters. We use kernel size 3 to capture trigrams (the most common local n-gram unit in NLP).

Why Conv before attention, not after? Placing convolution before attention means the attention module receives locally-smoothed representations, making it easier to identify which tokens are relevant globally. Placing it after would apply local smoothing to already-contextualized representations, which is less motivated.

1.5.3 Results and Analysis

Conv+RoPE achieves **perplexity 481.18** vs RoPE alone at 481.29 — essentially identical. This suggests that at context length 512 and model size 256, attention already captures local patterns sufficiently. The convolutional layer is redundant because each attention head can already learn to attend locally. This finding is consistent with the literature: convolutional augmentation tends to help more in audio (Conformer) than text, and more at larger scales.

1.6 Full Results and Comparative Analysis

Model	Perplexity ↓	Parameters	Throughput (tok/s) ↑	Training Time (s)
Baseline	578.83	29,022,208	22,571	304.8
MQA	426.02	28,561,664	22,905	304.8
SWA (W=64)	587.46	29,022,208	22,958	305.1
Linear Attention	559.04	29,022,208	26,539	264.6
ALiBi	476.94	29,022,208	22,109	317.3
RoPE	481.29	29,022,208	22,066	319.2
Conv+RoPE	481.18	29,022,208	22,053	319.2

Table 1: Comparison of all model variants on WikiText-2 (3 epochs, context length 512)

Key findings:

- **Best perplexity:** MQA (426.02). Sharing K/V across heads may act as a regularizer on our small dataset, preventing overfitting.
- **Best throughput:** Linear Attention (26,539 tok/s, +18% over baseline). The $O(Td^2)$ complexity vs $O(T^2d)$ pays off at sequence length 512.

- **Best positional encoding:** ALiBi (476.94) narrowly beats RoPE (481.29). Both significantly outperform the baseline’s learned embeddings.
- **Conv hybrid:** Negligible improvement over RoPE alone, suggesting local patterns are already captured by attention at this scale.
- **SWA** is the only variant worse than baseline, confirming that WikiText-2 requires long-range context that a window of 64 cannot capture.

Training stability: All models converged smoothly over 3 epochs. MQA initially had a bug where the global optimizer variable was not updated, causing the model to not learn — this was fixed by explicitly reassigning the optimizer before training. No gradient explosions were observed (gradient clipping at norm 1.0 was applied throughout).

2 Sparsity & Optimization

2.1 Overview

Parameter-efficient fine-tuning (PEFT) methods adapt large pretrained models while updating only a small fraction of parameters. This section compares three methods — LoRA, AdaLoRA, and SoRA — on the CoLA (Corpus of Linguistic Acceptability) task from the GLUE benchmark. CoLA requires binary classification of whether a sentence is grammatically acceptable.

Backbone: We use BERT-base-uncased (110M parameters) rather than DeBERTa-v3-base as originally planned, because DeBERTa’s architecture caused consistent class collapse (MCC=0) due to interactions between the model’s disentangled attention and the PEFT modules. BERT provides a cleaner baseline for comparing PEFT methods, and the comparison remains valid since all three methods are applied to the same backbone.

Evaluation metric: Matthews Correlation Coefficient (MCC), which is the standard metric for CoLA. MCC handles class imbalance better than accuracy — CoLA has 70% grammatical sentences (class 1) and 30% ungrammatical (class 0), so a model predicting all-1s gets MCC=0.

2.2 Methods

2.2.1 LoRA

Motivation: Fine-tuning all parameters of a large model is expensive and prone to catastrophic forgetting. LoRA hypothesizes that the weight update ΔW needed for fine-tuning lies in a low-dimensional subspace, and approximates it with a low-rank decomposition.

Method: Freeze pretrained weights W_0 and learn:

$$W = W_0 + \Delta W = W_0 + BA$$

where $A \in R^{r \times d}$ is initialized randomly and $B \in R^{k \times r}$ is initialized to zero (so $\Delta W = 0$ at initialization). The rank $r \ll \min(k, d)$ controls the expressivity of the update. We apply LoRA to the query, key, and value projection matrices with $r = 8$, $\alpha = 16$.

Why Q, K, V? These projections are the most task-sensitive components of BERT — they determine what information is attended to. The feedforward layers are more task-agnostic.

2.2.2 AdaLoRA

Motivation: LoRA uses a fixed rank r for all layers, but not all layers require the same capacity for adaptation. Some layers may need more rank (more adaptation) while others need less.

Method: AdaLoRA parameterizes the weight update using SVD-style decomposition $\Delta W = P\Lambda Q$ where Λ is a diagonal matrix of singular values. During training, singular values are pruned based on importance scores, effectively reducing rank in less important layers and increasing it in more important ones. The total parameter budget is fixed.

Why AdaLoRA? It addresses the fixed-rank limitation of LoRA by learning where to allocate parameters, potentially improving efficiency.

2.2.3 SoRA

Motivation: SoRA (Sparse Low-Rank Adaptation) extends LoRA by adding an explicit sparsity mechanism through gating. Rather than fixed rank, a gating vector g controls which rank components are active, and ℓ_1 regularization drives unnecessary components to zero.

Method: The weight update becomes:

$$\Delta W = B \cdot \text{diag}(g) \cdot A$$

with objective:

$$\mathcal{L}(\Delta) = \mathcal{L}_0(\Delta) + \lambda \sum_k \|g^{(k)}\|_1$$

The gating vector is optimized using proximal gradient descent, which applies soft-thresholding to exactly zero out small gate values. We implement SoRA from scratch since it is not available in HuggingFace PEFT.

Custom implementation details: Our SoRA implementation adds LoRA-style layers to BERT’s attention projections and applies the proximal update after each gradient step via a custom `Trainer` subclass.

2.3 Results

Method	Best MCC	Trainable Params	% of Total	Training Time (s)
LoRA ($r = 8$)	0.513	443,906	0.40%	314.9
AdaLoRA	0.410	665,522	0.60%	319.1
SoRA (custom)	0.236	296,642	0.27%	186.3

Table 2: Comparison of PEFT methods on CoLA (BERT-base-uncased, 3 epochs)

2.4 Analysis

LoRA achieves the best MCC (0.513) with only 0.40% of parameters trained. The fixed rank of 8 is sufficient for CoLA, which is a relatively simple binary classification task.

AdaLoRA underperforms LoRA (MCC 0.410) despite using more parameters (0.60%). This is likely because AdaLoRA’s rank pruning requires more epochs to converge — the importance scores need time to stabilize before meaningful pruning occurs. With only 3 epochs, AdaLoRA may be spending too much capacity on rank allocation rather than task learning. With more training, AdaLoRA would likely match or exceed LoRA.

SoRA achieves the lowest MCC (0.236) but uses the fewest parameters (0.27%) and trains fastest (186.3s). The lower performance is expected for two reasons: (1) our implementation applies SoRA only to a subset of layers compared to LoRA/AdaLoRA; (2) proximal updates gradually drive gates to zero, requiring more epochs to reach full convergence. Notably, all effective ranks remained at 8/8 after 3 epochs, confirming that the sparsity-inducing effect had not yet fully taken hold.

Parameter efficiency vs performance tradeoff: LoRA offers the best balance. SoRA is the most parameter-efficient but needs more training time to converge. AdaLoRA is the least efficient in this short-training regime.

2.5 SGD vs Proximal Gradient Descent (Q2)

2.5.1 Derivation of SGD Update Rule

The SoRA objective with respect to gating vector $g^{(k)}$ is:

$$\mathcal{L}(g^{(k)}) = \mathcal{L}_0(g^{(k)}) + \lambda \|g^{(k)}\|_1$$

The ℓ_1 norm $\|g\|_1 = \sum_i |g_i|$ is not differentiable at $g_i = 0$. Its subgradient is:

$$\partial|g_i| = \begin{cases} \{1\} & g_i > 0 \\ \{-1\} & g_i < 0 \\ [-1, 1] & g_i = 0 \end{cases}$$

The standard choice is $\partial|g_i| = \text{sign}(g_i)$ (with $\text{sign}(0) = 0$). The SGD update is then:

$$g^{(k)} \leftarrow g^{(k)} - \eta \left(\nabla_g \mathcal{L}_0 + \lambda \cdot \text{sign}(g^{(k)}) \right)$$

For the gating-only update (fixing other parameters):

$$\boxed{g^{(k)} \leftarrow g^{(k)} - \eta \lambda \cdot \text{sign}(g^{(k)})}$$

2.5.2 Proximal Soft-Thresholding Update

The proximal operator for the ℓ_1 penalty solves:

$$\text{prox}_{\eta\lambda\|\cdot\|_1}(g) = \arg \min_v \frac{1}{2}\|v - g\|^2 + \eta\lambda\|v\|_1$$

The closed-form solution is the soft-thresholding operator:

$$g^{(k)} \leftarrow \text{sign}(g^{(k)}) \cdot \max(|g^{(k)}| - \lambda\eta, 0)$$

2.5.3 Mathematical Comparison

The key difference is whether values reach *exactly* zero:

SGD: Given $g_i = 0.001$, $\eta = 0.1$, $\lambda = 0.05$:

$$g_i \leftarrow 0.001 - 0.1 \times 0.05 \times 1 = 0.001 - 0.005 = -0.004$$

The sign flips and the value oscillates around zero rather than reaching it.

Proximal: Same values:

$$g_i \leftarrow \text{sign}(0.001) \times \max(0.001 - 0.005, 0) = 1 \times 0 = 0$$

The value is set to exactly zero.

Our numpy and PyTorch implementations confirm this empirically: given $g = [0.5, -0.3, 0.01, -0.001, 0.8]$, proximal produces **1 exact zero** while SGD produces **0 zeros**.

2.5.4 Does Subgradient Choice Matter?

Yes, the choice of subgradient at $g_i = 0$ affects the theoretical comparison. At $g_i = 0$, any value in $[-1, 1]$ is a valid subgradient of $|g_i|$. Common choices:

- $\partial|g_i| = 0$: No update at zero — the value stays at zero once it reaches it, but it rarely reaches exactly zero with SGD.
- $\partial|g_i| = \text{sign}(g_i) = 0$: Same as above.
- $\partial|g_i| \in [-1, 1]$ random: Noisy updates that can push values away from zero.

In practice, SGD with any subgradient choice cannot guarantee exact zeros because floating point arithmetic prevents convergence to exactly zero without a thresholding operation. The proximal operator, by contrast, applies thresholding explicitly and always produces exact zeros for values within the threshold. This is the fundamental advantage of proximal methods for sparsity-inducing regularization.

2.6 Low-Rank Adaptation for xLSTM and Mamba (Q3)

2.6.1 Motivation

LoRA was designed for Transformer attention matrices. Applying it to recurrent architectures (xLSTM) and state space models (Mamba) requires identifying the analogous weight matrices that control information flow and are most likely to benefit from task-specific adaptation.

2.6.2 xLSTM Adaptation

xLSTM extends LSTM with matrix memory (mLSTM) and scalar memory (sLSTM) cells. The key weight matrices are the gate matrices: input gate W_i , forget gate W_f , output gate W_o , and cell update W_c .

Why gate matrices? These matrices control what information enters, persists, and exits the cell at each timestep — exactly the parameters that need task-specific adaptation. The forget gate W_f in particular controls long-term memory retention, which varies significantly across tasks.

Proposed adaptation:

$$W_i = W_i^0 + B_i A_i, \quad W_f = W_f^0 + B_f A_f$$

We apply LoRA to W_i and W_f (input and forget gates) as they have the most influence on information flow. W_o and W_c are kept frozen as secondary parameters.

SoRA extension: For SoRA-style adaptation, gating vectors g_i and g_f are added:

$$W_i = W_i^0 + B_i \cdot \text{diag}(g_i) \cdot A_i$$

The proximal update then automatically determines which rank components are needed for each gate, potentially discovering that the forget gate needs more rank than the input gate for certain tasks.

Implementation results: Our simplified xLSTM+LoRA achieves **1,088 trainable parameters out of 17,472 total (6.2%)**, demonstrating the parameter efficiency of the approach.

2.6.3 Mamba Adaptation

Mamba is a selective state space model (SSM) where the state transition depends on the input. The core computation is:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t$$

where $\bar{A}$ is discretized from A , and B, C are input-dependent projections.

Why B and C? B controls how the current input updates the hidden state (analogous to the value projection in attention), and C controls how the hidden state maps to output (analogous to the output projection). A encodes the core dynamics of the state space and is better left unchanged.

Proposed adaptation:

$$B = B^0 + B_b B_a, \quad C = C^0 + C_b C_a$$

Methodological difference from Transformers: In Transformers, LoRA is applied to static weight matrices. In Mamba, B and C are *input-dependent* — they change with each input token. This means the LoRA adaptation $B_b B_a$ is added to a dynamic projection, which is a subtle but important difference. The adaptation must be compatible with Mamba’s selective scan operation.

Implementation results: Our simplified Mamba+LoRA achieves **640 trainable parameters out of 2,768 total (23.1%)**.

Relevant prior work: BlockMamba (Mestha et al., ICLR 2026) from SAILDL explores structured sparsity for Mamba, which is complementary to our low-rank adaptation approach. Combining BlockMamba-style sparsity with LoRA adaptation of B and C is a natural direction for future work.

2.7 Limitations and Future Work

Due to compute and time constraints, the following planned experiments were not completed:

- **Context length sweep:** Evaluating all attention variants at 512, 1024, 2048, and 4096 tokens as specified. This would show how each variant scales with sequence length.
- **Extrapolation test:** Training on $L_{train} = 512$ and evaluating on $L_{test} = 1024$ and $L_{test} = 2048$ to measure positional encoding generalization. ALiBi and RoPE are theoretically expected to degrade less than learned embeddings due to their relative position encoding.
- **Peak GPU memory profiling:** Systematic memory measurement across all variants at different context lengths.

- **Combined experiment:** Evaluating Conv hybrid with best attention variant (MQA) combined with best positional encoding (ALiBi).

These are the natural next steps for a more complete empirical study. The core architectural comparisons and findings remain valid within the completed experiments.